

BEDROCK ARCHITECTURE

Bedrock Language-Neutral ABI

ELF and Linking ABI

Version 0.1

Contents

1	Scope and Profiles	1
1.1	ABI Hierarchy	1
1.2	ABI Profiles	1
2	Terminology	2
3	ELF Object Format	3
3.1	ELF Identification Bytes	3
3.2	ELF Header Sizes	3
3.3	Entry Point Semantics	3
3.4	Program Header Policy	4
3.5	Section and Symbol Policy	4
3.6	Bedrock Object Attributes	4
4	Data Register Banking	6
4.1	Architectural ABI Model	6
4.2	DBANK-neutral Boundaries	6
4.3	Object Attribute Rules	6
4.4	Recommended Uses	7
5	Program Loading	8
5.1	Loading Rules	8
5.2	Page Permissions	8
5.3	Execution Environment	8
5.4	Initial Non-Segment State	9
6	Sections and Symbols	10
6.1	Standard Sections	10
6.2	Symbol Model	10
7	Relocation Set	11
7.1	Expression Model	11
7.2	Expression Terms	11
7.3	GOT and PLT Model	11
7.4	Relocations	12

7.5	Relocation Uses	13
8	Dynamic Linking	15
8.1	Dynamic Linking Rules	15
8.2	GOT Reserved Entries	15
8.3	PLT Layout	15
9	Thread-Local Storage	16
9.1	TLS Base Model	16
9.2	Model Selection	16
9.3	TLSDESC Entry	16
9.4	TLSDESC Call ABI	16
9.5	TLSDESC Relocation and Relaxation	17
10	Code Models	18
10.1	low Model Details	18
10.2	small Model Details	19
10.3	medium Model Details	19
10.4	high Model Details	19
10.5	large Model Details	19
11	Assembler Contract	21
11.1	Strict Mode	21
11.2	Relaxation	21

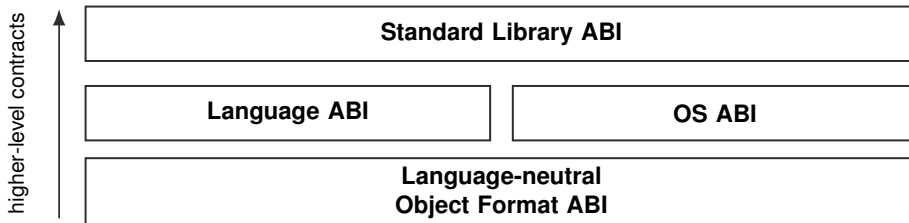
List of Tables

1	Reference ABI Profiles	1
3	ELF Identification	3
5	ELF e_ident Values	3
7	ELF Header Size Values	3
9	ELF Entry Point Rules	4
11	ELF Program Header Rules	4
13	ELF Section and Symbol Rules	4
15	Bedrock ELF Attribute Notes	5
17	Data Register Bank Model	6
19	DBANK Object Attribute Rules	6
21	ELF Loading Rules	8
23	ELF Segment Permission Mapping	8
25	Loader Execution Environment	8
27	Loader Initial Non-Segment State	9
29	Standard Section Names	10
31	Symbol Classes	10
33	Relocation Expression Model	11
35	Relocation Expression Terms	11
37	Bedrock ELF Relocation Types	12
39	Dynamic Linking Rules	15
41	Global Offset Table Reserved Entries	15
43	Procedure Linkage Table Layout	15
45	TLS Base Model	16
47	TLSDESC Entry Layout	16
49	TLSDESC Resolver Call ABI	17
51	Bedrock Code Models	18
53	low Code Model Details	18
55	small Code Model Details	19
57	medium Code Model Details	19
59	high Code Model Details	19
61	large Code Model Details	20

1 Scope and Profiles

The Bedrock language-neutral ABI defines the object, linking, loading, relocation, dynamic linking, TLS, and code-model contracts used by conforming assemblers, linkers, loaders, and language runtimes. It does not define a language calling convention, C type layout, process entry stack, or operating-system argument protocol.

1.1 ABI Hierarchy



Current document. This document specifies the language-neutral object-format layer. It fixes the ELF, relocation, linking, loading, TLS, dynamic-linking, and code-model contracts consumed by language ABIs, OS ABIs, and standard-library ABIs.

1.2 ABI Profiles

Table 1. Reference ABI Profiles

Profile	Meaning
bedrock-elf	Conforming Bedrock ELF64 objects using the reference relocation, symbol, section, linking, and loading model.

2 Terminology

These terms are used by the language-neutral ELF ABI. ISA-level address and register terms keep the meanings defined by the architecture manual.

language-neutral ABI

The ELF, relocation, dynamic-linking, TLS, and code-model contract that is shared by language bindings and operating-system profiles.

OS ABI

The operating-system profile layered above this document. It defines process entry, argument vectors, environment state, loader policy, signal or exception delivery policy, and any OS-specific segment setup.

pre-segment virtual address

The address value used by ELF symbols, relocations, ordinary function addresses, and ordinary data pointers before segment pre-translation.

load base

The runtime address bias applied to a loaded ET_DYN object before its symbol values and relative relocations are interpreted.

ABI-visible linkage domain

The region in which ordinary ELF code addresses are near-call entry points and do not carry an explicit CS value.

near-call entry address

A pre-segment code address that can be reached with ordinary CALL and returned from with ordinary RET inside the current linkage domain.

veneer

A near-call entry point supplied by a linker, loader, runtime, or OS ABI to hide a more complex transfer sequence from ordinary ABI callers.

code model

A set of placement and relocation assumptions used by code generation to choose direct displacement forms, GOT/PLT forms, or full-width address materialization.

active data-register bank

The D-register bank selected by the DBANK architectural selector. An ordinary Dn operand names D[DBANK][n] while the selector is active.

DBANK-neutral boundary

An ABI boundary that is entered and left with DBANK equal to zero, so ordinary ABI code observes the bank-zero D0-D7 register set.

3 ELF Object Format

Table 3. ELF Identification

Field	Value
Format	ELF64
ELF Class	ELFCLASS64
Data Encoding	ELFDATA2LSB
File Types	ET_REL, ET_EXEC, ET_DYN
Machine	EM_BEDROCK (0xffb0)
e_flags	0
Unknown nonzero e_flags	reject
Extension metadata	.note.bedrock
Relocation Encoding	RELA
Relocation Addend	explicit addend
String Table	.strtab
Symbol Table	.symtab
Relocation Prefix	.rela

3.1 ELF Identification Bytes

Table 5. ELF e_ident Values

e_ident Field	Value
EI_MAG	0x7f 'E' 'L' 'F'
EI_CLASS	ELFCLASS64
EI_DATA	ELFDATA2LSB
EI_VERSION	EV_CURRENT
EI_OSABI	ELFOSABI_NONE
EI_ABIVERSION	0

3.2 ELF Header Sizes

Table 7. ELF Header Size Values

Header Field	Bytes
e_ehsize	64
e_phentsize	56
e_shentsize	64

3.3 Entry Point Semantics

Table 9. ELF Entry Point Rules

File Type	e_entry Meaning
ET_REL	0
ET_EXEC	absolute pre-segment virtual byte address
ET_DYN (executable)	load-bias-relative pre-segment virtual byte address; runtime entry is load base + e entry
ET_DYN (shared object without entry)	0

3.4 Program Header Policy

Table 11. ELF Program Header Rules

Field	Rule
Supported p_type	PT_LOAD PT_DYNAMIC PT_INTERP PT_NOTE PT_TLS
PF_X	1
PF_W	2
PF_R	4
p_align PT_LOAD	page size or larger; minimum 4096

3.5 Section and Symbol Policy

Table 13. ELF Section and Symbol Rules

Field	Rule
section types	standard ELF section types
section flags	SHF_WRITE SHF_ALLOC SHF_EXECINSTR SHF_TLS
st_info	standard ELF binding and type
st_other	architecture-specific bits reserved; must be zero
ET_REL	section-relative byte offset
linked_image	pre-segment virtual byte address

3.6 Bedrock Object Attributes

Table 15. Bedrock ELF Attribute Notes

Attribute	Encoding	Default	Meaning
BEDROCK_ATTR_REQUIRED_DBANK_COUNT	unsigned integer	1	Minimum number of architectural D-register banks required by this object. Objects that never select a nonzero DBANK record 1. Loaders may reject an image when CPUID reports fewer implemented banks than the maximum required value across the linked image.

4 Data Register Banking

Bedrock implementations may provide multiple architectural D-register banks. The data-register banking instructions are part of the base ISA; CPUID reports how many banks are implemented, and ELF attributes record whether a linked image requires more than bank zero. The language-neutral ABI preserves bank zero as the ordinary ABI-visible D0-D7 register set.

4.1 Architectural ABI Model

Table 17. Data Register Bank Model

Item	Rule
selector	DBANK
selector width bits	4
architectural namespace	16 banks
bank zero	ABI-visible D0-D7
ordinary d operand	Dn names D[DBANK][n]
discovery	CPUID DATA REGISTER BANKS leaf

4.2 DBANK-neutral Boundaries

- Linked ELF entry points, PLT entries, TLSDESC resolvers, IFUNC resolvers, and language ABI function boundaries are DBANK-neutral unless a higher layer defines a private convention.
- A DBANK-neutral boundary is entered with DBANK = 0 and must return or tail-transfer with DBANK = 0.
- Nonzero bank contents are volatile across DBANK-neutral boundaries.
- Bank-zero D0-D7 preservation and argument rules are defined by the active language ABI, not by this language-neutral object ABI.
- Code may use nonzero banks inside a function, veneer, or runtime-private region when it restores DBANK before crossing a DBANK-neutral boundary.

4.3 Object Attribute Rules

Table 19. DBANK Object Attribute Rules

Rule	Value
required attribute	BEDROCK_ATTR_REQUIRED_DBANK_COUNT
default required count	1
explicit bank use	An object that names DBn explicitly must require at least $n + 1$ banks. Code that selects DBANK from a register operand cannot be inferred precisely from syntax; the producer must record an attribute large enough for every runtime-selected bank that may be used.
dynamic bank select	
link rule	The linked image requirement is the maximum required DBANK count across all input objects.

Rule	Value
load rule	A loader may reject an image when CPUID reports fewer implemented banks than the linked image requires.

4.4 Recommended Uses

- spill avoidance inside hot leaf regions
- loop-local accumulators for REP or REPG lowering
- JIT or interpreter register caching
- runtime-private veneers that restore DBANK before ordinary ABI transfer

5 Program Loading

Bedrock ELF program loading follows the ordinary ELF segment model. The language-neutral ABI defines executable mapping, loader-visible object rules, and the minimum execution environment needed to begin running the image; language runtimes and OS ABIs define their own entry argument protocol above this layer.

5.1 Loading Rules

Table 21. ELF Loading Rules

Rule	Value
executable file types	ET_EXEC, ET_DYN
segment alignment	page size
minimum page size	4096
PT_INTERP	supported
bss zero fill	zero bytes through file-size to memory-size gap
bss zero fill alignment	8

5.2 Page Permissions

Table 23. ELF Segment Permission Mapping

ELF Flags	PTE Flags	Meaning
PF_R	P, U	readable user page
PF_R PF_W	P, U, W	readable writable user page
PF_R PF_X	P, U, X	readable executable user page
PF_R PF_W PF_X	P, U, W, X	readable writable executable user page

5.3 Execution Environment

Table 25. Loader Execution Environment

Requirement	Rule
code fetch	The active code context must permit instruction fetch at the ELF entry address.
data access	The active data context must permit ordinary data access to mapped loadable data regions.
stack access	The active stack context must permit stack reads and writes for the stack supplied by the OS ABI or loader profile.
ordinary linkage domain	Executable images and ordinary shared objects are linked as near-call objects in one ABI-visible linkage domain. ELF function addresses are pre-segment code addresses, not CS:PC pairs.
far segment crossing	Segment-changing calls are outside the ordinary ELF dynamic-linking contract. Runtimes or OS ABIs may hide such crossings behind near-call veneers.
code data stack context	CS, DS, SS, and their descriptor values are not fixed by the language-neutral ABI.

5.4 Initial Non-Segment State

Table 27. Loader Initial Non-Segment State

Register	Initial Value
FLAGS	0
FSTATUS	0
FFLAGS	0

6 Sections and Symbols

Bedrock uses ordinary ELF sections. The names below are the standard section conventions for conforming Bedrock ELF objects.

6.1 Standard Sections

Table 29. Standard Section Names

Section	Attributes
.text	alloc, execute, read
.rodata	alloc, read
.data	alloc, read, write
.bss	alloc, read, write, zero fill
.init	alloc, execute, read
.fini	alloc, execute, read
.got	alloc, read, write
.got.plt	alloc, read, write
.plt	alloc, execute, read
.dynamic	alloc, dynamic
.dynsym	alloc, dynamic symbol table
.dynstr	alloc, dynamic string table
.rela.dyn	alloc, relocation
.rela.plt	alloc, relocation
.note.bedrock	note

6.2 Symbol Model

Table 31. Symbol Classes

Item	Values
Bindings	local, global, weak
Visibility	default, hidden, protected
Types	notype object function ifunc section file

- Function symbols name the first instruction of the callable entry point.
- Object symbols name the first byte of the object storage.
- ELF st value stores a byte offset relative to the symbol's defining section.

7 Relocation Set

Bedrock ELF objects use RELA relocation entries with explicit addends. The Bedrock ABI defines how each relocation type patches instruction or data fields.

7.1 Expression Model

Table 33. Relocation Expression Model

Rule	Value
canonical form	<code>symbol + addend</code>
address space	pre-segment virtual byte addresses unless a relocation explicitly names a TLS offset or descriptor
section relative allowed	true
absolute allowed	true
pc relative base	address of relocation field
overflow policy	error

7.2 Expression Terms

Table 35. Relocation Expression Terms

Term	Meaning
S	Runtime byte address of the symbol named by the relocation after symbol resolution.
A	Explicit addend stored in the ELF64 Rela entry.
P	Runtime byte address of the relocation field being patched.
SB	Runtime byte address of the beginning of the symbol's defining section.
B	Runtime load base address of the loaded object that owns the relocation.
GB	Runtime byte address of the global offset table base for the loaded object.
GOT(S)	Runtime byte address of the GOT slot allocated for symbol S.
PLT(S)	Runtime byte address of the PLT entry allocated for symbol S.
TLS(S)	Byte offset of TLS symbol S from the ABI TLS base in GS0.
TLSDDESC(S)	Runtime byte address of the TLSDDESC entry allocated for TLS symbol S.
Resolver(X)	Runtime byte address returned by calling the resolver function at address X.

7.3 GOT and PLT Model

- The global offset table stores 64-bit addresses used by position-independent code and by dynamic symbol binding.
- GOT and PLT function addresses are near-call entry addresses in the ABI-visible linkage domain.
- A GOT relocation that names a symbol allocates or references one GOT slot for that symbol in the loaded object.
- The procedure linkage table contains executable entries used for external function calls through dynamic binding.

- A PLT relocation that names a function symbol targets the function's PLT entry instead of the final resolved function address.
- Dynamic GOT and PLT relocations are applied by the dynamic loader before control reaches code that depends on the resolved slot.

7.4 Relocations

Table 37. Bedrock ELF Relocation Types

Type	Name	Size	Calculation
0	R_BEDROCK_NONE	0	none
1	R_BEDROCK_ABS8	8-bit unsigned	$S + A$
2	R_BEDROCK_ABS16	16-bit unsigned	$S + A$
3	R_BEDROCK_ABS32	32-bit unsigned	$S + A$
4	R_BEDROCK_ABS64	64-bit unsigned	$S + A$
5	R_BEDROCK_PCREL16	16-bit signed	$S + A - P$
6	R_BEDROCK_PCREL32	32-bit signed	$S + A - P$
7	R_BEDROCK_PCREL64	64-bit signed	$S + A - P$
8	R_BEDROCK_WORD_PCREL16	16-bit word-scaled signed	$(S + A - P) / 2$
9	R_BEDROCK_WORD_PCREL32	32-bit word-scaled signed	$(S + A - P) / 2$
10	R_BEDROCK_IMM16	16-bit	$S + A$
11	R_BEDROCK_IMM32	32-bit	$S + A$
12	R_BEDROCK_IMM64	64-bit	$S + A$
13	R_BEDROCK_DISP16	16-bit signed	$S + A$
14	R_BEDROCK_DISP32	32-bit signed	$S + A$
15	R_BEDROCK_DISP64	64-bit signed	$S + A$
16	R_BEDROCK_SECTION_REL32	32-bit unsigned	$S + A - SB$
17	R_BEDROCK_SECTION_REL64	64-bit unsigned	$S + A - SB$
18	R_BEDROCK_CALL_TARGET	variable-size	$S + A$
19	R_BEDROCK_JMP_TARGET	variable-size	$S + A$
20	R_BEDROCK_LONG_CONTROL_TARGET	variable-size	$S + A$
21	R_BEDROCK_GOT64	64-bit unsigned	$GOT(S) + A$
22	R_BEDROCK_GOTPCREL32	32-bit signed	$GOT(S) + A - P$
23	R_BEDROCK_GOTPCREL64	64-bit signed	$GOT(S) + A - P$
24	R_BEDROCK_GOTOFF32	32-bit signed	$S + A - GB$
25	R_BEDROCK_GOTOFF64	64-bit signed	$S + A - GB$
26	R_BEDROCK_GOT_BASE_PCREL32	32-bit signed	$GB + A - P$
27	R_BEDROCK_GOT_BASE_PCREL64	64-bit signed	$GB + A - P$
28	R_BEDROCK_PLT32	32-bit signed	$PLT(S) + A - P$
29	R_BEDROCK_PLT64	64-bit unsigned	$PLT(S) + A$
30	R_BEDROCK_RELATIVE	64-bit unsigned	$B + A$
31	R_BEDROCK_GLOB_DAT	64-bit unsigned	$S + A$
32	R_BEDROCK_JUMP_SLOT	64-bit unsigned	$S + A$
33	R_BEDROCK_COPY	variable-size unsigned	copy S bytes
34	R_BEDROCK_IRELATIVE	64-bit unsigned	$Resolver(B + A)$
35	R_BEDROCK_TLS_OFFSET32	32-bit signed	$TLS(S) + A$
36	R_BEDROCK_TLS_OFFSET64	64-bit signed	$TLS(S) + A$

Type	Name	Size	Calculation
37	R_BEDROCK_TLSDESC_GOTPCREL32	32-bit signed	$\text{TLSDESC}(S) + A - P$
38	R_BEDROCK_TLSDESC_GOTPCREL64	64-bit signed	$\text{TLSDESC}(S) + A - P$
39	R_BEDROCK_TLSDESC_CALL	0	none
40	R_BEDROCK_TLSDESC	128-bit unsigned	tlsdesc_init

7.5 Relocation Uses

- R_BEDROCK_NONE: no relocation
- R_BEDROCK_ABS8: absolute data or immediate field
- R_BEDROCK_ABS16: absolute data, immediate, or descriptor field
- R_BEDROCK_ABS32: absolute data or compact absolute EA payload
- R_BEDROCK_ABS64: absolute data, pointer, or full absolute EA payload
- R_BEDROCK_PCREL16: relative branch or address materialization
- R_BEDROCK_PCREL32: relative branch or address materialization
- R_BEDROCK_PCREL64: full-width relative address materialization
- R_BEDROCK_WORD_PCREL16: branch displacement encoded in instruction words
- R_BEDROCK_WORD_PCREL32: branch displacement encoded in instruction words
- R_BEDROCK_IMM16: symbolic immediate payload
- R_BEDROCK_IMM32: symbolic immediate payload
- R_BEDROCK_IMM64: symbolic immediate payload
- R_BEDROCK_DISP16: EA displacement payload
- R_BEDROCK_DISP32: EA displacement payload
- R_BEDROCK_DISP64: EA displacement payload
- R_BEDROCK_SECTION_REL32: offset from the start of the defining section
- R_BEDROCK_SECTION_REL64: offset from the start of the defining section
- R_BEDROCK_CALL_TARGET: CALL target
- R_BEDROCK_JMP_TARGET: JMP or Jcc target
- R_BEDROCK_LONG_CONTROL_TARGET: segment-changing control transfer target
- R_BEDROCK_GOT64: absolute address of a GOT slot
- R_BEDROCK_GOTPCREL32: PC-relative address of a GOT slot
- R_BEDROCK_GOTPCREL64: full-width PC-relative address of a GOT slot
- R_BEDROCK_GOTOFF32: symbol offset from GOT base
- R_BEDROCK_GOTOFF64: full-width symbol offset from GOT base
- R_BEDROCK_GOT_BASE_PCREL32: PC-relative materialization of GOT base

- `R_BEDROCK_GOT_BASE_PCREL64`: full-width PC-relative materialization of GOT base
- `R_BEDROCK_PLT32`: PC-relative call or branch to PLT entry
- `R_BEDROCK_PLT64`: absolute address of a PLT entry
- `R_BEDROCK_RELATIVE`: dynamic load-base adjustment
- `R_BEDROCK_GLOB_DAT`: dynamic GOT data symbol resolution
- `R_BEDROCK_JUMP_SLOT`: dynamic PLT function symbol resolution
- `R_BEDROCK_COPY`: dynamic executable copy relocation
- `R_BEDROCK_IRELATIVE`: dynamic indirect resolver relocation
- `R_BEDROCK_TLS_OFFSET32`: GS0-relative TLS offset
- `R_BEDROCK_TLS_OFFSET64`: full-width GS0-relative TLS offset
- `R_BEDROCK_TLSDESC_GOTPCREL32`: PC-relative address of TLSDESC entry
- `R_BEDROCK_TLSDESC_GOTPCREL64`: full-width PC-relative address of TLSDESC entry
- `R_BEDROCK_TLSDESC_CALL`: marks a TLSDESC resolver call for linker relaxation
- `R_BEDROCK_TLSDESC`: dynamic loader writes resolver/function and argument words for TLS symbol

8 Dynamic Linking

Dynamic objects use a fixed Bedrock GOT and PLT contract so that linkers, loaders, and position-independent code agree on lazy binding and resolver state.

8.1 Dynamic Linking Rules

Table 39. Dynamic Linking Rules

Rule	Value
lazy binding	allowed
ifunc	always supported
copy relocations	supported
irelative relocations	supported
ordinary shared object calls	near CALL/RET within the ABI-visible linkage domain
global offset table symbol	<code>_GLOBAL_OFFSET_TABLE_</code>
plt entry alignment	16
plt entry size	32
plt0 size	32

8.2 GOT Reserved Entries

Table 41. Global Offset Table Reserved Entries

Index	Name	Meaning
0	<code>_DYNAMIC</code>	dynamic section address
1	<code>loader object handle</code>	dynamic loader object handle for this loaded object
2	<code>lazy_resolver</code>	dynamic loader lazy binding entry

8.3 PLT Layout

Table 43. Procedure Linkage Table Layout

Entry	Effect
PLT0	receives D0=relocation index and A0=GOT base, then transfers to GOT[2]
PLTn_step0	indirect branch through GOTPLT slot n
PLTn_step1	unresolved slot path sets D0=relocation index
PLTn_step2	unresolved slot path sets A0=GOT base and branches to PLT0

9 Thread-Local Storage

Thread-local storage is addressed through the GS0 segment register. TLS references that remain inside the program and have a link-time constant offset use the local-exec model. TLS references that cross dynamic object boundaries, may be interposed, or otherwise require runtime resolution use the TLSDESC model.

9.1 TLS Base Model

Table 45. TLS Base Model

Rule	Value
tls base register	GS0
bounds only	disabled
segment bounds only bit	0
addressing	TLS offsets are translated by GS0 segment pre-translation before page-table translation.
local exec offset base	GS0 base
program internal model	local_exec
dynamic tls model	TLSDESC
local exec relocations	R_BEDROCK_TLS_OFFSET32 R_BEDROCK_TLS_OFFSET64
dynamic tls relocations	R_BEDROCK_TLSDESC_GOTPCREL32 R_BEDROCK_TLSDESC_GOTPCREL64 R_BEDROCK_TLSDESC_CALL R_BEDROCK_TLSDESC

9.2 Model Selection

- A TLS symbol defined in the same program image may use local-exec addressing when its byte offset from GS0 is link-time constant.
- A TLS reference from a shared object or to an interposable TLS symbol uses TLSDESC.
- The linker may relax a TLSDESC sequence to local-exec when symbol binding and final placement make the GS0-relative offset constant.

9.3 TLSDESC Entry

Table 47. TLSDESC Entry Layout

Offset	Field	Type	Meaning
+0x00	resolver or fast function	u64	function that resolves the descriptor or a relaxed fast-path implementation
+0x08	argument	u64	resolver argument or relaxed fast-path payload

9.4 TLSDESC Call ABI

Table 49. TLSDESC Resolver Call ABI

Rule	Value
input	A0 = address of TLSDESC entry
output	D0 = signed byte offset from GS0 TLS base
clobbers	caller-saved registers only

9.5 TLSDESC Relocation and Relaxation

- TLSDESC GOTPCREL relocations materialize the address of the descriptor entry.
- The TLSDESC call relocation marks the descriptor call instruction so the linker may replace the call sequence when relaxation is legal.
- The TLSDESC descriptor relocation initializes the two-word descriptor entry used by the dynamic loader.

10 Code Models

Code models define placement and locality assumptions that assemblers, linkers, and compilers may use when selecting address materialization, displacement, GOT, PLT, and relocation forms. Bedrock 32-bit displacement forms are signed, so direct 32-bit relative and base-relative references cover a -2 GiB to +2 GiB - 1 window rather than an unsigned 4 GiB window.

Table 51. Bedrock Code Models

Model	Placement	Direct Range	Rule
low	low static address space	0x00000000..0x7fffffff	Static image model for absolute addresses that fit the nonnegative signed 32-bit range. Signed 32-bit absolute, immediate, and displacement forms may directly name objects in this range.
small	local signed 32-bit relative window	-2GiB..+2GiB-1 from the use site or selected base	Position-independent local model where direct code, nearby data, GOT, and PLT references fit signed 32-bit PC-relative or base-relative displacements.
medium	local code and GOT signed 32-bit window	code, GOT, and PLT within -2 GiB..+2 GiB - 1	Code, GOT, and PLT references fit signed 32-bit relative displacements, but data objects may be outside that range and are normally addressed through GOT-resolved 64-bit addresses.
high	high static address space	internal references within signed 32-bit displacement window	Static high-address image model. The image is placed in a high virtual-address region, while internal code and data references are expected to fit signed 32-bit displacements relative to the use site, GOT, or an implementation-defined image base. Final high addresses are not assumed to be directly encodable in signed 32-bit absolute payloads.
large	full 64-bit address space	none	No placement assumption is made; full-width address materialization and 64-bit relocation forms are used by default.

10.1 low Model Details

Table 53. low Code Model Details

Aspect	Rule
code references	absolute or relative 32-bit
data references	absolute or base-relative 32-bit
GOT/PLT	not required
default relocations	R_BEDROCK_ABS32 R_BEDROCK_IMM32 R_BEDROCK_DISP32 R_BEDROCK_PCREL32
relaxation	may shrink direct branches or address materialization when the final target fits a shorter encoding.

10.2 small Model Details

Table 55. small Code Model Details

Aspect	Rule
code references	PC-relative 32-bit
data references	PC-relative or GOTPCREL 32-bit
GOT/PLT	within signed 32-bit relative range
default relocations	R_BEDROCK_PCREL32 R_BEDROCK_GOTPCREL32 R_BEDROCK_PLT32 R_BEDROCK_TLSDESC_GOTPCREL32
relaxation	may shrink 32-bit relative forms to shorter encodings, or relax GOT/TLSDESC sequences when the resolved symbol is local and in range.

10.3 medium Model Details

Table 57. medium Code Model Details

Aspect	Rule
code references	PC-relative 32-bit
data references	GOT-resolved 64-bit address
GOT/PLT	within signed 32-bit relative range
default relocations	R_BEDROCK_PCREL32 R_BEDROCK_GOTPCREL32 R_BEDROCK_GOT64 R_BEDROCK_PLT32 R_BEDROCK_TLSDESC_GOTPCREL32
relaxation	may relax GOT data references to direct 32-bit relative references when the final data object is in range.

10.4 high Model Details

Table 59. high Code Model Details

Aspect	Rule
code references	PC-relative 32-bit or image-base relative
data references	PC-relative GOT or image-base relative when in range
GOT/PLT	image-local or base-relative
default relocations	R_BEDROCK_PCREL32 R_BEDROCK_GOTPCREL32 R_BEDROCK_ABS64 R_BEDROCK_DISP64
relaxation	may use compact internal references when the final reference is within the signed 32-bit window; full-width or base-relative sequences remain valid.

10.5 large Model Details

Table 61. large Code Model Details

Aspect	Rule
code references	full-width or long control transfer
data references	full-width address materialization
GOT/PLT	full width
default relocations	R_BEDROCK_ABS64
	R_BEDROCK_PCREL64
	R_BEDROCK_GOTPCREL64
	R_BEDROCK_PLT64
	R_BEDROCK_TLSDESC_GOTPCREL64
relaxation	may shrink full-width forms after final placement proves that a shorter form is in range.

11 Assembler Contract

Default ABI: bedrock-elf

11.1 Strict Mode

- Pure ISA syntax is accepted.
- Ambiguous symbolic operands require an explicit relocation annotation.
- Noncanonical encodings are rejected.

11.2 Relaxation

- Short and long branches may be selected automatically when the target is known.
- Overlong encodings are emitted only when explicitly requested.